

1. Explain the difference between frontend, backend, and full-stack development with suitable real-world examples.

Frontend Development

- Deals with the visual part of a website or app that users interact with.
- Uses HTML, CSS, JavaScript, and frameworks like React, Angular, or Vue.
- Focuses on design, layout, and user experience (UI/UX).
- Example: On Amazon, the product listings, buttons, and navigation bar you see and click are frontend.

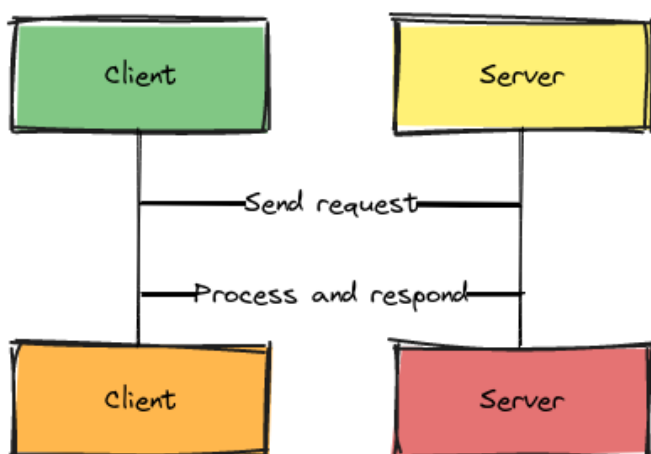
Backend Development

- Handles the server-side logic, data storage, and communication between the server and database.
- Uses languages like Java, Python, Node.js, or PHP, and databases like MySQL or MongoDB.
- Responsible for authentication, data processing, and server management.
- Example: When you log into Netflix, the backend verifies your credentials and fetches your watchlist.

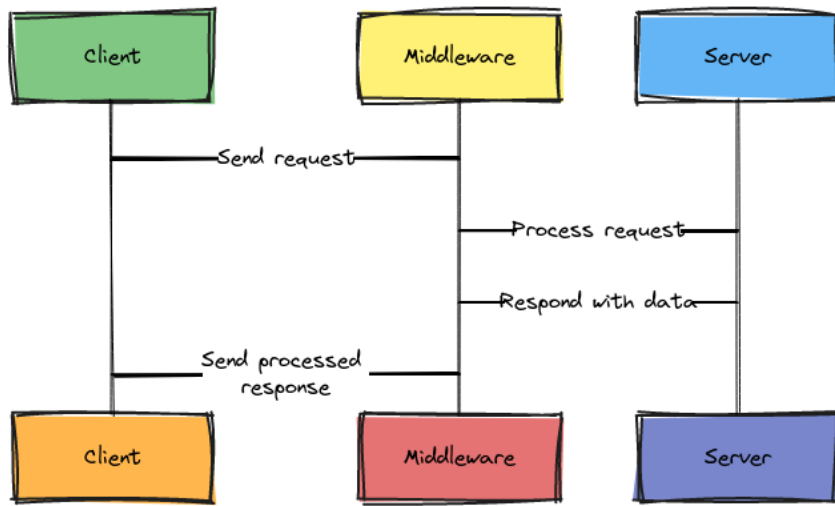
Full-Stack Development

- Involves working on both frontend and backend.
- Full-stack developers can build the entire application—from the user interface to database handling.
- Common stacks include MERN (MongoDB, Express, React, Node) and MEAN (MongoDB, Express, Angular, Node).
- Example: A full-stack developer can design an online store's web pages and also build the system that stores and manages orders.

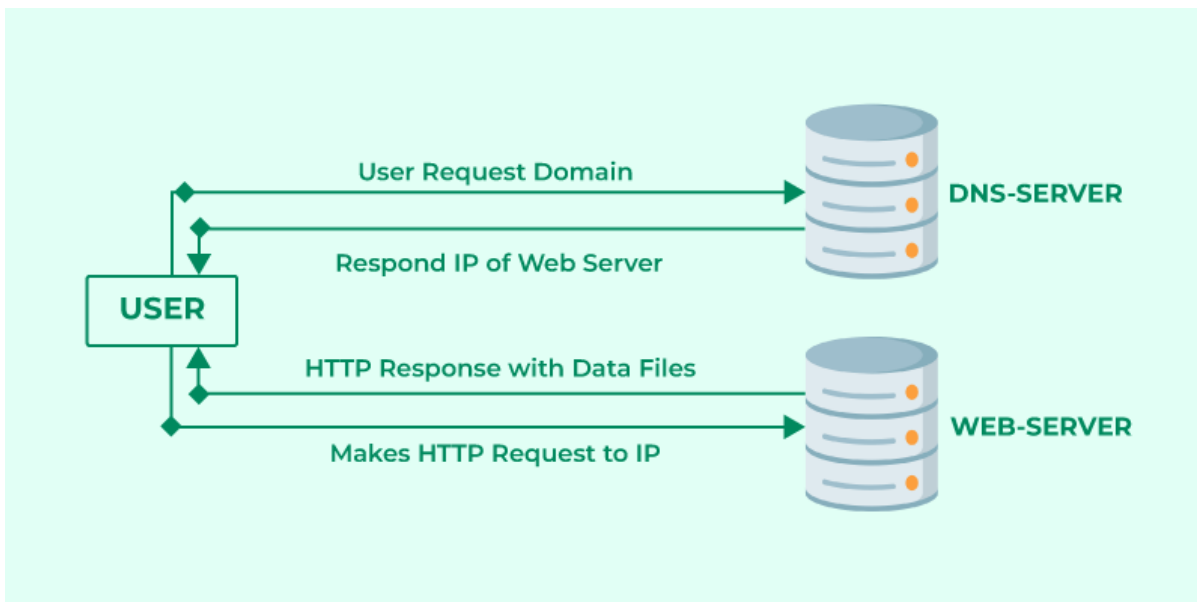
2. Create a simple diagram showing how the client-server model works in web architecture.



Two tier client server architecture



Three tier client server architecture



Client Server Request and Response

3. Describe how a browser requests and displays a web page from a web server.

The user enters a website URL (like `www.example.com`) into the browser's address bar.

The browser converts the domain name into an IP address using the **DNS (Domain Name System)**.

The browser then sends an **HTTP or HTTPS request** to the web server at that IP address, asking for the web page.

The **web server** receives the request, processes it, and sends back the requested files (HTML, CSS, JavaScript, images, etc.) as an **HTTP response**.

The browser downloads these files and starts **rendering** the page.

The HTML is read first, which may link to CSS and JavaScript files.

CSS defines how elements should look (style and layout), and JavaScript adds interactivity.

Finally, the browser displays the complete web page to the user.

4. Identify and list the tools required to set up a web development environment. Explain the purpose of each.

1. Code Editor or IDE

- Used to write and edit HTML, CSS, JavaScript, and backend code.
- Examples: Visual Studio Code, Sublime Text, Atom.
- Provides syntax highlighting, debugging, and auto-completion features for efficient coding.

2. Web Browser

- Used to test and view web pages during development.
- Examples: Google Chrome, Mozilla Firefox, Microsoft Edge.
- Includes developer tools to inspect elements, debug JavaScript, and analyze performance.

3. Version Control System (Git)

- Tracks changes in code, allows collaboration, and maintains project history.
- Example: Git, with hosting platforms like GitHub or GitLab.
- Helps developers revert to previous versions and work in teams.

4. Local Web Server

- Simulates a real server environment for testing web applications locally.
- Examples: XAMPP, WAMP, or Node.js.
- Runs backend code and helps test server-side functionality before deployment.

5. Package Manager

- Manages installation of libraries, frameworks, and dependencies.
- Examples: npm (for Node.js), pip (for Python).
- Simplifies adding external modules to a project.

6. Database Management System (DBMS)

- Used to store, retrieve, and manage data for web applications.
- Examples: MySQL, MongoDB, PostgreSQL.
- Provides structured storage and easy access to application data.

7. Frameworks and Libraries

- Provide pre-written code to speed up development.
- Examples: React, Angular, Bootstrap for frontend; Express.js, Django for backend.
- Helps maintain clean, reusable, and efficient code.

8. Testing Tools

- Ensure the website or app works correctly across devices and browsers.
- Examples: Postman (for API testing), Selenium (for automation).

- Helps identify and fix bugs early in development.

5. Explain what a web server is and give examples of commonly used servers.

Web Server

- A web server is a software or hardware system that **stores, processes, and delivers web pages** to users over the internet.
- When a browser sends a request for a webpage, the web server **retrieves the requested files (HTML, CSS, images, etc.)** and sends them back to the browser using **HTTP or HTTPS protocols**.
- It can also run **server-side scripts** (like PHP, Node.js, or Python) to generate dynamic content before sending it to the client.

Functions of a Web Server

- Handles client requests and sends appropriate responses.
- Manages multiple requests simultaneously.
- Provides security features like HTTPS and authentication.
- Supports load balancing and logging for performance tracking.

Examples of Commonly Used Web Servers

- **Apache HTTP Server** – One of the most widely used open-source web servers.
- **Nginx** – Known for high performance and ability to handle many concurrent connections.
- **Microsoft IIS (Internet Information Services)** – A web server developed by Microsoft for Windows systems.
- **LiteSpeed** – A high-performance server known for speed and scalability.
- **Node.js** – Often used as a lightweight, event-driven web server for JavaScript-based applications.

6. Define the roles of a frontend developer, backend developer, and database administrator in a project.

Frontend Developer

- Designs and implements the **user interface (UI)** of the website or application.
- Writes code using **HTML, CSS, JavaScript**, and frameworks like React or Angular.
- Ensures the website is **responsive, visually appealing, and user-friendly**.
- Works closely with designers to translate mockups into working web pages.

Backend Developer

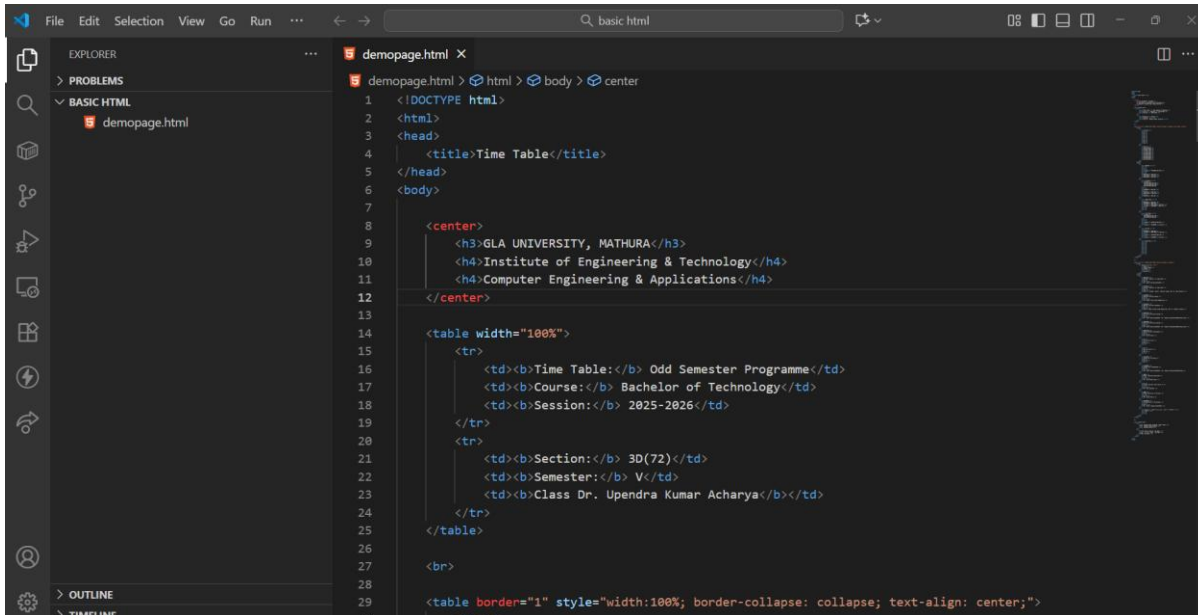
- Builds and maintains the **server-side logic** of the application.
- Uses languages like **Java, Python, Node.js, or PHP** to handle requests and process data.
- Creates **APIs** and connects the frontend with databases.
- Ensures **security, authentication, and performance** of the server.

Database Administrator (DBA)

- Designs, manages, and maintains the **database** used in the project.

- Ensures **data integrity, security, and backup**.
- Optimizes database performance and handles queries efficiently.
- Works with backend developers to provide necessary data for the application.

7. Install VS Code and configure it for HTML, CSS, and JavaScript development. Take a screenshot of the setup.



8. Explain the difference between static and dynamic websites. Provide an example of each.

Static Website

- Contains **fixed content** that does not change unless manually updated.
- Built using **HTML, CSS, and sometimes JavaScript**.
- Simple to develop and loads **fast** because content is pre-written.
- Does **not interact with databases** or server-side scripts.
- Example: A company's **portfolio website** showing information about services and team.

Dynamic Website

- Content **changes automatically** based on user interaction or other data.
- Built using **server-side languages** like PHP, Python, Node.js, along with databases.
- Can display personalized content, handle forms, and manage user accounts.
- Example: **Facebook** or **Amazon**, where content updates based on user actions and database information.

9. Research and list five web browsers. Explain how rendering engines differ between them.

1. Google Chrome

- Uses the **Blink** rendering engine.
- Blink focuses on **speed, standards compliance, and performance**, allowing Chrome to render modern web pages quickly.

2. Mozilla Firefox

- Uses the **Gecko** rendering engine.
- Gecko emphasizes **customization, privacy, and standards compliance**, rendering pages accurately but sometimes slower than Blink.

3. Microsoft Edge

- Uses **Blink** (Edge moved from its old EdgeHTML to Chromium-based Blink).
- Offers similar performance to Chrome but integrates **Windows features** and security options.

4. Safari

- Uses the **WebKit** rendering engine.
- WebKit is optimized for **Apple devices**, focusing on smooth performance, energy efficiency, and accurate rendering on iOS/macOS.

5. Opera

- Uses **Blink**, like Chrome, since it switched to Chromium.
- Supports fast rendering and includes **built-in features** like ad blockers and VPN.

Difference in Rendering Engines

- Rendering engines determine **how HTML, CSS, and JavaScript are processed and displayed**.
- Blink (Chrome, Edge, Opera) is designed for **speed and modern web compatibility**.
- Gecko (Firefox) focuses on **accuracy and standards compliance**.
- WebKit (Safari) is optimized for **Apple devices**, balancing speed and efficiency.
- Each engine can slightly **interpret CSS and JavaScript differently**, which is why websites may look or behave differently across browsers.

10. Draw a labelled diagram showing the basic web architecture flow — client, server, database, and APIs.

